AI Assistant

How It Works — End-to-End Guide

Ingestion -> knowledge graph -> hybrid retrieval -> agent -> evaluation -> training -> MLflow -> deployment

Internal guide for new maintainers • July 2026 • local-first, fully self-hosted

1. What this system is

A fully local enterprise assistant that answers engineering questions from **two sources at once**: a **Neo4j knowledge graph** built from Jira / Confluence / GitLab exports (vector + multi-hop graph retrieval), and the **live** Jira / Confluence / GitLab APIs. A LangGraph agent routes each question, a local LLM served by **Ollama** writes the answer, and an evaluation pipeline (**Hammer**) scores every response, mines training data, and gates fine-tuned adapter deployments — a closed, self-improving loop.

Where things run. The serving stack is containerized (Docker Compose): **Neo4j**, **MongoDB**, the **FastAPI backend**, and the **React frontend**. **Ollama** runs on the host for GPU access. The **ingestion** and **training/Hammer** steps are run on demand from their own Python environments — they are not part of the container and are never run at the same time as each other on a single GPU.

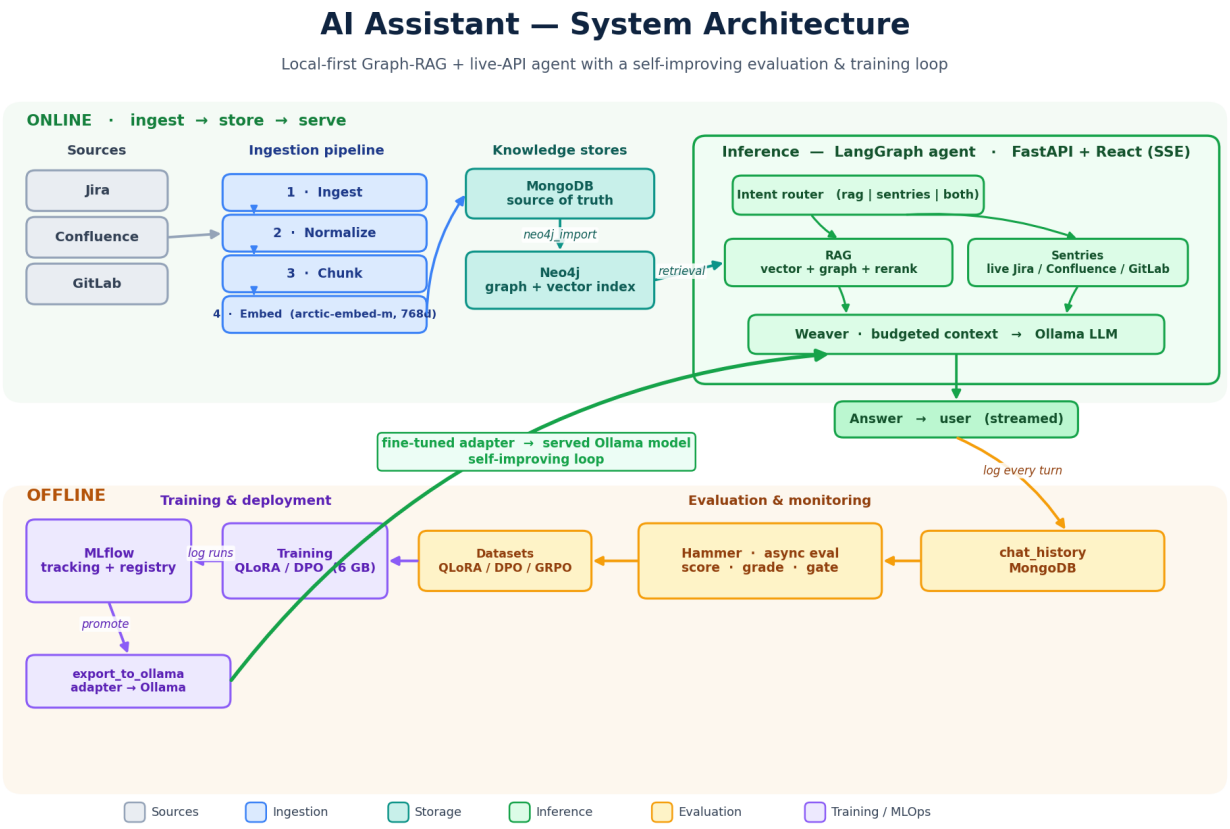

Figure 1. End-to-end architecture — ingestion builds the graph; the agent serves from graph + live APIs; Hammer evaluates every answer and mines training data; QLoRA/DPO runs are tracked in MLflow; the fine-tuned adapter returns to the served Ollama model (self-improving loop).

2. The lifecycle at a glance

Data flows one way into the graph, then answers flow back out as training signal:

Phase	What happens	Lands in
Ingest	Pull Jira/Confluence/GitLab, normalize, chunk, embed	MongoDB
Graph	Import chunks + entities + typed links; build vector index	Neo4j
Serve	Intent routing -> hybrid retrieval + live APIs -> Ollama answer	User + chat_history
Evaluate	Hammer scores answers, grades them, mines datasets	MongoDB + JSONL
Train	QLoRA / DPO on mined data, tracked in MLflow	Adapter + MLflow
Deploy	Export adapter onto the served Ollama model	Ollama (new tag)

3. Stage 1 — Ingestion

`pipeline/run_pipeline.py` runs the whole knowledge-base build end to end. Each step is a separate module so you can re-run any stage in isolation.

Step	Module	Transformation
1 Ingest	pipeline.ingestion_newest	GitLab + Jira + Confluence -> raw docs in MongoDB
2 Normalize	pipeline.normalize_data	raw docs -> normalized_documents
3 Chunk	pipeline.chunk_documents	normalized -> document_chunks
4 Embed	pipeline.embed	chunks -> 768-dim vectors (arctic-embed-m) in MongoDB
5 Graph	rag.neo4j_import	MongoDB -> Neo4j property graph + vector index

MongoDB is the source of truth; Neo4j is rebuilt from it. Run the whole thing, or skip stages you have already done:

```
# full build (reads .env for Mongo + Neo4j connection)
python -m pipeline.run_pipeline

# skip flags (env vars): SKIP_INGEST=true SKIP_EMBED=true
```

4. Stage 2 — The knowledge graph

`rag/neo4j_import.py` turns the embedded chunks into a rich property graph. `IMPORT_SOURCE=mongo` (default) reads MongoDB; `jsonl` reads pre-embedded `DATA/embedded_*.jsonl` files.

Nodes	Chunk • JiraTicket • ConfluencePage • GitLabDoc • Project • Space • User • Component • Version • Label
Structure	(Chunk)-[:PART_OF]->(JiraTicket|ConfluencePage|GitLabDoc); Confluence page hierarchy via CHILD_OF / IN_SPACE
Typed Jira links	20+ relationship types: BLOCKS, BLOCKED_BY, RELATES_TO, CAUSES, DUPLICATES, IMPLEMENTS, SPLIT_FROM, IS_TESTED_BY, CLONES, …
Metadata edges	IN_PROJECT, ASSIGNED_TO (User), HAS_COMPONENT, FIX_IN (Version), HAS_LABEL
Vector index	768-dim cosine index over Chunk embeddings (EMBEDDING_DIM=768) — the ANN entry point for retrieval

This graph is what makes retrieval *hybrid*: a vector hit on a chunk can be expanded along real Jira dependencies and Confluence hierarchy, not just semantic similarity.

5. Stage 3 — Hybrid retrieval

`rag/neo4j_search.py` answers a query in three moves:

- **Vector ANN** over chunk embeddings (arctic-embed-m), with the project filter pushed down into the Cypher query.
- **Graph expansion** from each seed chunk along typed Jira links and the Confluence hierarchy, with a distance-decayed prior (closer hops rank higher).
- **Cross-encoder rerank** of the merged vector+graph pool (RERANKER_MODEL) — the single biggest precision win — with a parent-document diversity cap so one chatty ticket can't dominate.

A question naming a ticket key (e.g. `ABC-1234`) skips search entirely with a direct indexed graph fetch. Vector hits carry `issue_key / priority / assignee / status` so the model can cite grounded metadata. Tunable via `SEARCH_TOP_K, GRAPH_HOPS, RERANK_POOL, PARENT_DOC_CAP`.

```
python -m rag.neo4j_search hybrid 'who is blocking the payments migration?'
```

6. Stage 4 — The agent at query time

`agent/intent_agent.py` is a **LangGraph** state machine exposed by the FastAPI backend (`api/main.py`, SSE streaming) and driven by the React UI.

Node	Role
intent classifier	Keyword/regex router (no LLM latency on the hot path), optional LLM fallback; routes to rag | sentries | both
rag_node	Hybrid retrieval, then a score filter (RAG_MIN_SCORE) and a source cap (RAG_MAX_SOURCES)
sentries_node	Live Jira / Confluence / GitLab API calls in parallel, with per-project fan-out caps
weaver	Budgets each prompt section (KB / live data / history) so the window can never silently overflow, then streams the answer from Ollama (OLLAMA_MODEL)

Every turn is written to MongoDB `chat_history` (fire-and-forget, off the hot path). That log is the raw material Hammer evaluates in the next stage.

7. Stage 5 — Evaluation (Hammer)

`hammer/run_hammer.py` runs **asynchronously alongside** production and never touches the inference graph. It scores logged answers, grades them, and turns the good ones into training data.

Command	What it does
score	Run the evaluator on unscored chat_history (RAGAS faithfulness/relevance + a zero-LLM validator for live-data answers)
check	Rolling quality check — alerts if scores regress
dataset	Export training sets: qlora / dpo / grpo / seed / all
benchmark	Run the adapter gate (score delta + per-intent faithfulness floors) before a deploy
status / watch	Print full status, or run score+check continuously every N hours

Answers are graded **GOLD / SILVER / BRONZE / FAILED** with orthogonal failure tags. Only clean, high-grade examples flow into the QLoRA/DPO pools — the gate is what keeps fine-tuning from training on its own mistakes.

```
python hammer/run_hammer.py score
python hammer/run_hammer.py dataset all
```

```
python hammer/run_hammer.py status
```

8. Stage 6 — Training (QLoRA / DPO) + MLflow

`training/pipeline.py` fine-tunes a small base model on the mined data, entirely on a 6 GB consumer GPU. Three stages:

Stage	What it does
prepare	Pull scored data from MongoDB, deduplicate, split, format to the production prompt shape, snapshot
train	QLoRA SFT, DPO, or sequential (SFT then DPO). Saves a LoRA adapter; logs params/metrics/artifacts to MLflow
promote	Register the adapter in the MLflow model registry (a version is marked Production)

Why it fits 6 GB. QLoRA loads the base in 4-bit and only trains small adapters:

Base model (4-bit)	~4.0 GB
LoRA adapters	~0.1 GB
Optimizer states	~0.2 GB
Activations (grad-checkpointing)	~1.2 GB
Total	~5.5 GB — fits an RTX 3050 6 GB

MLflow is the system of record for training: every run's hyper-parameters, metrics, and the adapter artifact are tracked (`MLFLOW_TRACKING_URI`, e.g. `sqlite:///mlflow.db` with `mlruns/`), and the model registry holds the versioned adapters and which one is Production. Browse it with `mlflow ui`.

```
python training/pipeline.py prepare
python training/pipeline.py train --method qlora # or dpo / sequential
python training/pipeline.py promote <run_id>
mlflow ui # inspect runs + registry
```

9. Stage 7 — Put the fine-tune back into inference

This is the step that closes the loop. Inference serves through **Ollama**, but training produces a **PEFT LoRA adapter** on a Hugging Face base — which Ollama cannot load directly. `training/export_to_ollama.py` bridges them by adding the trained weight onto the very same Ollama model named in your inference `.env`:

```
# adapter -> GGUF -> ollama create (FROM $OLLAMA_MODEL + ADAPTER)
python -m training.export_to_ollama --adapter training/export/final_adapter --name
weaver-ft

# then point inference at it and restart the backend:
# OLLAMA_MODEL=weaver-ft
```

The one rule — alignment. A LoRA only loads into the architecture it was trained on, so the training base must match the model you serve (serve `qwen3:8b` -> train the `Qwen3-8B` HF base). Set `HF_MODEL_ID` accordingly; the built-in `check_base_model_compat()` warns you if they diverge.

10. Operating on a single 6 GB GPU

Inference and training each fit 6 GB *alone* but not together, so they run from separate, isolated Python environments and are never triggered at the same time.

- During a query the GPU already holds **two** consumers: the Ollama model *and* the backend's embedding + reranker models.
- A training run is a third, ~5.5 GB consumer. Overlapping it with a live query can exhaust 6 GB (OOM).
- **Rule of thumb:** don't start a training run while you are actively serving inference (and vice-versa). On a larger GPU this restriction relaxes.

Because the venvs are isolated (each installs its own torch), the serving stack keeps its CUDA image while Hammer/training run independently — the two never contend except for raw VRAM, which the no-overlap rule handles.

11. Command cheat-sheet

Task	Command
Start the stack (GPU)	docker compose up -d --build
Start the stack (no GPU)	docker compose -f docker-compose.yml -f docker-compose.cpu.yml up -d --build
Build the knowledge base	python -m pipeline.run_pipeline
Import into Neo4j only	docker compose exec backend python -m rag.neo4j_import
Search the graph	python -m rag.neo4j_search hybrid 'your query'
Score answers (Hammer)	python hammer/run_hammer.py score
Mine training data	python hammer/run_hammer.py dataset all
Train	python training/pipeline.py train --method qlora
Promote adapter	python training/pipeline.py promote <run_id>
Ship fine-tune to Ollama	python -m training.export_to_ollama --adapter ... --name weaver-ft

Open the app at `http://localhost:3000`. Neo4j browser is at `http://localhost:7474`. Configure everything through `.env` (copy from `.env.example`).